

UNION ALL Views

#sql-server #performance #views #optimizer

What They Are

A UNION ALL view combines multiple tables into a single logical entity:

```
Create View vImgImage As
  Select * From ImgImage           -- 10,532,809 rows
  Union All
  Select * From ImgImage_IntId     -- 0 rows
```

These views typically exist for migration or partitioning reasons. The application was designed to eventually split data across tables (for example, moving from varchar IDs to int IDs), and the view provides a stable interface so application code doesn't need to change.

The Performance Impact

Even when one of the underlying tables is empty, the UNION ALL view introduces overhead.

1. Plan Complexity

The optimizer must generate a plan that accesses both tables. For each filter predicate on the view, the optimizer must push the predicate down into each branch of the UNION ALL. This doubles the number of seek/scan operators in the plan. With an empty table, the second branch completes instantly, but it still adds plan compilation cost and plan cache bloat.

2. Predicate Pushdown Limitations

The optimizer can push simple WHERE predicates through a UNION ALL view (called predicate pushdown). But complex scenarios interfere:

- JOIN predicates from outer queries may not push cleanly through the view expansion.
- OR conditions spanning the view and other tables are especially problematic.
- TOP with ORDER BY over a UNION ALL requires the optimizer to sort-merge both branches even if one is empty.

3. Statistics Confusion

Each underlying table has its own statistics. The optimizer must combine them to estimate cardinality for the view as a whole. With skewed data or very different distributions between tables, these combined estimates can be significantly off.

4. Interaction with Other Anti-Patterns

UNION ALL views amplify other performance problems:

- Combined with **LEFT JOIN OR Anti-Pattern**: the optimizer must consider both view branches against both join paths, which is four access paths.
- Combined with correlated subqueries (see **Correlated Subqueries to CTEs**): each subquery invocation expands into two table accesses.
- Combined with TOP subqueries: the rowgoal optimization may not apply correctly across the view boundary.

When to Replace the View with the Base Table

Replace vImgImage with ImgImage directly when:

- The second table has zero rows AND no application is writing to it.
- The migration that motivated the view is complete (or abandoned).
- You need every bit of plan simplification for a critical query.

Keep the view when:

- The second table may receive data in the future (active migration in progress).
- Other procs or application code depend on the view name.
- The performance gain from bypassing the view is marginal compared to other fixes.

The Conservative Approach

If you're unsure whether the view is still needed, keep it but fix the surrounding query patterns. The UNION refactoring of a **LEFT JOIN OR Anti-Pattern** gives the optimizer a much better chance of pushing predicates through the view cleanly, because each branch of your UNION is a simple INNER JOIN rather than a complex multi-table LEFT JOIN with OR filters.

```
-- This pushes well through a UNION ALL view:
Select i.Id, i.ContentCode
From vImgImage i
Join RxAssoc IR On i.Id = IR.ImgId
Where i.ArchivedDtTm Is Null

-- This pushes poorly:
Select i.Id, i.ContentCode
From vImgImage i
Left Join RxAssoc IR On i.Id = IR.ImgId
Left Join CanAssoc IC On i.Id = IC.ImgId
Where (i.Id = IR.ImgId Or i.Id = IC.ImgId)
```

Real-World Example

In lsp_ImgGetListOfTopXImagesToMove, the vImgImage view over ImgImage (10.5M rows) and ImgImage_IntId (0 rows) was the base of a query with the **LEFT JOIN OR Anti-Pattern**. The combination of view expansion plus LEFT JOINS plus OR predicates produced 1,141,351 average logical reads per execution. The materialize-once + EXISTS refactoring (Fix #2 in **LEFT JOIN OR Anti-Pattern**) kept vImgImage in place but reshaped the surrounding query to push predicates through the view cleanly. Reads dropped to about 214K per execution.

Identifying UNION ALL Views

Views with the v prefix (like vImgImage, v0e0rder) are common in enterprise systems. When you see unexpectedly high reads on a seemingly simple query, check the view definition:

```
-- Check if a view is a UNION ALL view
Select definition
From sys.sql_modules
Where object_id = Object_Id('vImgImage')
```

Related Concepts

- **LEFT JOIN OR Anti-Pattern**: dramatically amplified by UNION ALL views.

- **FORCESEEK Hints:** often applied to force seeks through view branches.
- **Query Store Triage:** views don't appear separately in Query Store. You see the expanded query.